

LUMINA PROTOCOL

Parametric DeFi Insurance for AI Agents

Version: 1.0

Year: 2026

Network: Base L2 (Chain 8453)

Settlement: USDC

Website: <https://www.lumina-org.com>

GitHub: <https://github.com/org-lumina>

Contact: labs@lumina-org.com

This document describes the architecture, mechanisms, and security properties of the Lumina Protocol — the first parametric insurance protocol designed exclusively for autonomous AI agents operating in DeFi.

Disclaimer

This whitepaper is for informational purposes only and does not constitute financial advice, an offer to sell, or a solicitation to buy any securities or tokens. The Lumina Protocol is experimental software deployed on public blockchains. Users should understand the risks involved in DeFi protocols, including smart contract risk, oracle risk, and market risk. Lumina Protocol products are structured as parametric derivative contracts, not traditional insurance policies. Please conduct your own research before interacting with the protocol.

Table of Contents

- 1** Abstract
- 2** Introduction: The Problem
- 3** Lumina Protocol: The Solution
- 4** Architecture Overview
- 5** Insurance Products
- 6** Vault System & Liquidity
- 7** Monetization & Unit Economics
- 8** Oracle System & Trigger Flow
- 9** Security Architecture
- 10** Risk Management
- 11** API & Agent Integration
- 12** Legal & Regulatory Framework
- 13** Governance
- 14** Roadmap
- 15** Contract Addresses
- 16** References

1. Abstract

Lumina Protocol is a parametric insurance protocol built exclusively for AI agents operating in decentralized finance (DeFi) on Base L2. Unlike traditional insurance that requires human claims adjusters and subjective assessment, Lumina uses objective on-chain data from Chainlink oracles to automatically verify trigger conditions and resolve claims — enabling fully autonomous, machine-to-machine (M2M) insurance.

The protocol offers four insurance products covering the primary risks that AI agents face in DeFi: Black Swan events (extreme price drops), stablecoin depegs, impermanent loss, and smart contract exploits. Liquidity is provided by human LPs who deposit USDC into four isolated vaults with different risk-return profiles, earning yield from premiums and Aave V3 lending.

Key innovations include: (1) parametric trigger conditions that eliminate subjective claims assessment, (2) an API-first architecture where policies are purchased programmatically via REST endpoints, (3) a multisig oracle system (N-of-M) with no single point of failure, (4) soulbound vault shares preventing flash-loan manipulation, (5) a session-based buyer consent mechanism protecting agent wallets, and (6) a TimelockController with 48-hour delay governed by a Gnosis Safe multisig.

The protocol is deployed on Base mainnet with 13 verified contracts, 79 passing tests, and settlement in USDC with yield generation via Aave V3.

2. Introduction: The Problem

2.1 The Rise of Autonomous AI Agents

The DeFi ecosystem is experiencing a paradigm shift. Autonomous AI agents — software programs that operate wallets, execute trades, provide liquidity, and manage portfolios without human intervention — are becoming significant participants in on-chain markets. These agents operate 24/7, execute strategies across multiple protocols, and manage substantial capital.

However, these agents face the same risks as human traders: flash crashes, stablecoin depegs, impermanent loss in liquidity pools, and smart contract exploits. Unlike humans, agents cannot evaluate insurance terms, file claims, or navigate dispute processes. They need insurance that is purely programmatic — purchasable via API, triggered by objective data, and settled automatically.

2.2 Limitations of Existing Insurance

Current DeFi insurance protocols (Nexus Mutual, InsurAce, Ensuro) are designed for human users. They require manual claims filing, subjective assessment, and web-based interfaces. None offer a pure API for programmatic policy purchase, and none use parametric trigger conditions that can be verified autonomously.

Feature	Nexus Mutual	InsurAce	Ensuro	Lumina
Target User	Humans	Humans	Humans	AI Agents
Purchase	Web UI	Web UI	Web UI	REST API
Claims	Token vote	Committee	Automated	Parametric
Settlement	NXM/ETH	INSUR	USDC	USDC
Oracle	N/A	N/A	Chainlink	Chainlink + Multisig
API-first	No	No	Partial	Yes
Agent-native	No	No	No	Yes

Table 1: Competitive comparison

2.3 Market Opportunity

The total value managed by AI agents in DeFi is projected to grow from \$2B in 2025 to over \$50B by 2028. As these agents manage larger capital, demand for programmatic risk management tools will grow proportionally. Lumina is positioned to capture this emerging market as the first protocol purpose-built for autonomous agent insurance.

3. Lumina Protocol: The Solution

3.1 Design Principles

- **Parametric, not discretionary.** Claims are triggered by objective, on-chain verifiable conditions. No human judgment required.
- **API-first.** Every interaction is available via REST API. The web interface is for human monitoring only.
- **Agent-only policy purchase.** Policies can only be purchased programmatically. This is the core differentiator.
- **USDC settlement.** All premiums, payouts, and deposits in USDC for stability.
- **Isolated risk pools.** Four separate vaults prevent cross-contamination.
- **Transparent pricing.** Deterministic formulas, published and verifiable.

3.2 Three Roles

Role	Actor	Interaction	Purpose
Role 1: AI Agent	Software	API only	Purchases insurance policies autonomously
Role 2: Agent Owner	Human	Web dashboard	Sets up agent, monitors policies, manages keys
Role 3: LP Investor	Human	Web dashboard	Deposits USDC into vaults, earns yield

3.3 End-to-End Flow

Agent Owner creates API key + funds agent wallet → **Agent** calls POST /api/v2/purchase → **API** signs EIP-712 quote → **Relayer** submits tx → **CoverRouter** transfers premium from agent wallet → **Vault** locks collateral → **PolicyManager** records policy → **Chainlink oracles** monitor trigger conditions → **Oracle multisig** signs claim report → **Shield** verifies signatures + executes payout → **Agent wallet** receives USDC

Figure 1: Complete policy lifecycle from purchase to claim resolution

4. Architecture Overview

4.1 Contract Architecture (13 Contracts)

Layer	Contracts	Purpose
Routing	CoverRouter (UUPS)	Entry point. Validates quotes, collects premiums, routes to shields.
Management	PolicyManager (UUPS)	Records policies, tracks status, waterfall vault selection.
Oracle	LuminaOracle, PhalaVerifier	Multisig verification (N-of-M), Chainlink feeds, sequencer checks.
Vaults	VolShort, VolLong, StableShort, StableLong	Collateral pools. Deposits into Aave V3 for base yield.
Shields	BSS, Depeg, ILIndex, Exploit	Product logic: triggers, payout calculations, deductibles.

4.2 Upgradeability & Governance

All core contracts use UUPS (Universal Upgradeable Proxy Standard). Upgrade authority is held by a TimelockController with a mandatory 48-hour delay, governed by a Gnosis Safe multisig (2-of-3). No single key compromise can result in immediate fund drainage. The 48-hour window gives LPs time to react to any proposed changes.

4.3 External Dependencies

Dependency	Purpose	Risk Mitigation
Chainlink (5 feeds)	Price data: ETH, BTC, USDC, USDT, DAI	TWAP, staleness checks, sequencer uptime
Aave V3	Base yield on idle vault capital	Battle-tested, audited protocol
Base L2	Settlement layer	Sequencer uptime check with 1h grace
USDC (Circle)	Settlement currency	Depeg monitoring via Chainlink feed

5. Insurance Products

Lumina offers four parametric products. All use objective, on-chain verifiable trigger conditions that eliminate the need for subjective claims assessment.

Product	Premium	Duration	Deductible	Trigger Condition	Vault
Black Swan (BSS)	650 bps	7-30d	20%	Price drop > threshold	VolatileShort
Depeg USDT	250 bps	14-365d	15%	USDT < \$0.85	StableShort/Long
Depeg DAI	250 bps	14-365d	12%	DAI < \$0.88	StableShort/Long
IL Protection	850 bps	14-90d	2%	IL > threshold (48h)	VolatileLong
Exploit Shield	400 bps	90-365d	10%	Dual trigger (Chainlink+Phala)	StableLong

Table 2: Complete product catalog

Each product is implemented as a Shield contract inheriting from BaseShield. The shield defines the trigger condition, deductible, and payout calculation specific to the risk category. Payout formula: **payout = coverageAmount × (1 - deductibleBps / 10000).**

6. Vault System & Liquidity

6.1 Four-Vault Architecture

Lumina uses four isolated vaults. Each has a different risk-return profile defined by its cooldown period and product categories. Isolation prevents a large claim in one category from affecting others.

Vault	Cooldown	Products	Risk Profile
VolatileShort	30 days	BSS	High risk, high yield
VolatileLong	90 days	IL Protection	Medium-high risk
StableShort	90 days	Depeg (USDT, DAI)	Medium risk
StableLong	365 days	Exploit Shield	Lower risk, longer lock

6.2 Soulbound Shares & Cooldown

LPs receive non-transferable (soulbound) shares. To withdraw: request → wait cooldown → complete. Soulbound design prevents flash-loan share manipulation, secondary market speculation, and dilution attacks. Cooldowns match maximum policy durations for each vault.

6.3 Yield Generation

Idle vault capital is deposited into Aave V3 on Base for lending yield. LP return = Aave base yield + insurance premiums - claims paid. Capital allocated to active policies remains in the vault but is locked and cannot be withdrawn until policies expire or claims resolve.

6.4 Solvency Guarantee

Capital is locked at the moment of policy purchase, not at claim time. When an agent buys a policy, the vault immediately allocates sufficient capital to cover the maximum potential payout. This capital cannot be withdrawn by LPs until the policy expires or the claim is executed. Pending payouts are tracked separately — the vault never releases collateral before a payout is complete.

7. Monetization & Unit Economics

7.1 Revenue Model

Lumina Protocol captures value through a transparent, fixed-rate fee model applied to every policy purchase:

Revenue Stream	Rate	Source	Destination
Protocol Fee	3% of premiums	Every policy purchase	Protocol treasury (fee receiver)
LP Yield Spread	Varies	Aave V3 lending + premiums	LP depositors (in-vault)

7.2 Unit Economics Example

Consider an agent purchasing BSS coverage of \$10,000 for 14 days at 650 bps:

Component	Calculation	Amount
Gross Premium	$\$10,000 \times 6.5\% \times (14/365)$	\$24.93
Protocol Fee (3%)	$\$24.93 \times 3\%$	\$0.75
Net Premium to Vault	$\$24.93 - \0.75	\$24.18
Vault Capital Locked	$\$10,000 \times (1 - 20\% \text{ deductible})$	\$8,000
LP Yield (if no claim)	$\$24.18 / \$8,000$	0.30% (14 days)

Table 3: Unit economics for a single BSS policy

7.3 LP Fee Structure

There are no deposit or withdrawal fees for LPs. LP returns come entirely from the premium spread (premiums collected minus claims paid) and Aave V3 base yield on idle capital. This aligns LP incentives with protocol health — LPs earn more when the protocol prices risk accurately and maintains solvency.

7.4 Protocol Scalability

Revenue scales linearly with total policy volume. As more agents purchase policies, the protocol collects more fees. The cost structure is primarily infrastructure (API hosting, oracle operations) which scales sub-linearly, creating improving margins at higher volumes.

8. Oracle System & Trigger Flow

8.1 Dual Oracle Architecture

Lumina combines Chainlink decentralized price feeds with a custom multisig verification layer. Chainlink provides reliable price data for five assets. LuminaOracle adds N-of-M multisig signing.

8.2 Parametric Trigger Flow (Step by Step)

The following describes exactly how a claim is triggered, verified, and paid — using Black Swan Shield as an example:

Step 1 — Monitoring: The oracle backend continuously reads the Chainlink ETH/USD price feed on Base L2. It checks every block for trigger conditions.

Step 2 — Trigger Detection: When ETH price drops below the policy threshold (e.g., -35% from strike price), the oracle flags the event. It reads the TWAP to confirm it is not a flash crash artifact.

Step 3 — Report Generation: The oracle generates a structured claim report containing: policyId, triggeredPrice, timestamp, chainlinkRoundId, and payoutAmount.

Step 4 — Multisig Signing: The report is hashed (EIP-712) and signed by N-of-M independent signers. Each signer verifies the Chainlink data independently before signing. Currently configured as 2-of-3.

Step 5 — On-Chain Verification: The packed signatures (concatenated 65-byte ECDSA) are submitted to the Shield contract. The contract calls LuminaOracle.verifyPackedMultisig() which recovers each signer address, verifies they are authorized signers, checks for duplicates, and validates the quorum.

Step 6 — Payout Execution: If verification passes, the Shield calculates the payout ($\text{coverageAmount} \times (1 - \text{deductible})$), instructs the Vault to transfer USDC to the agent wallet. Collateral is released only after the transfer completes.

Figure 2: Complete parametric trigger and settlement flow

8.3 Multisig Oracle Features

Feature	Description
Current Config	2-of-3 independent signers
Signature Format	Packed ECDSA (concatenated 65-byte, ordered by address)
On-chain Verification	verifyPackedMultisig() — duplicate and authorization checks
Signer Rotation	addSigner() / removeSigner() without contract redeployment
Quorum Adjustment	setRequiredSignatures() — upgradeable N-of-M
Backwards Compatible	Single-signer mode (1-of-1) supported

8.4 Safety Mechanisms

- **TWAP verification:** Prevents flash crash manipulation.
- **Sequencer uptime check:** 1-hour grace period after L2 outages.
- **Staleness checks:** Rejects data older than configured heartbeat.
- **Large payout delay:** Additional time delay for payouts above threshold.
- **Daily payout rate limit:** Caps total payouts per day.
- **Circuit breakers:** Extreme volatility pauses new policy purchases.

9. Security Architecture

Lumina Protocol implements defense-in-depth security across every layer of the stack. The protocol has undergone comprehensive internal security review with dual-auditor methodology (Claude Code Security + Gemini Pro cross-validation) and all identified items have been resolved.

9.1 Smart Contract Security

- **CEI Pattern:** All state-modifying functions follow Checks-Effects-Interactions to prevent reentrancy.
- **SafeERC20:** All token transfers use OpenZeppelin SafeERC20 for safe interaction with non-standard ERC20s.
- **Soulbound Shares:** Non-transferable vault shares prevent flash-loan-based share price manipulation.
- **Capital Locking:** Vault capital is locked at policy purchase and released only after payout execution.
- **Coverage Validation:** CoverRouter rejects zero-coverage and zero-duration policies.
- **Payout Delay Enforcement:** Large payouts require non-zero delay period.
- **Session-based Buyer Consent:** EIP-712 signed sessions protect agents from unauthorized spending.

9.2 Infrastructure Security

- **TimelockController (48h):** All administrative actions require a 48-hour delay.
- **Gnosis Safe (2-of-3):** Multisig governance — no single key can execute changes.
- **Key Separation:** Three independent keys for Deployer, Oracle, and Relayer roles.
- **Rate Limiting:** API endpoints protected against abuse (100/15min, 5/hr key creation, 30/min purchases).
- **CORS Restriction:** API accepts requests only from authorized origins.
- **Helmet Headers:** Standard web security headers on all API responses.
- **Error Sanitization:** Internal errors never exposed to consumers.
- **Nonce Manager:** Sequential nonce processing prevents race conditions.
- **HMAC-SHA256 API Keys:** Persistent, salted key hashing with configurable salt.

9.3 Testing

The protocol includes 79 automated tests written in Solidity using the Foundry framework, covering all contract interactions, edge cases, access control, multisig verification, session approvals, and payout

timing. Tests run on every code change with zero tolerance for failures.

10. Risk Management

Risk Category	Mitigation Strategy
Smart contract vulnerability	79 tests, dual-auditor review, UUPS with 48h timelock, Gnosis Safe
Oracle manipulation	Multisig (2-of-3), TWAP, staleness checks, sequencer uptime feed
Flash loan attack	Soulbound shares, decimalsOffset in vault math
Vault insolvency	Capital locked at purchase, no premature release, pending payout tracking
Key compromise	3 separate keys, Gnosis Safe, TimelockController — no instant access
L2 sequencer downtime	Sequencer uptime feed with 1-hour grace period
USDC depeg	Chainlink USDC/USD feed monitoring, circuit breakers
Extreme market conditions	Circuit breakers, daily payout limits, per-wallet caps (\$50K on Exploit)

11. API & Agent Integration

11.1 REST API

Endpoint	Method	Purpose	Auth
/api/v2/health	GET	Health check	None
/api/v2/keys/create	POST	Create API key	None
/api/v2/quote	GET	Get premium quote	API Key
/api/v2/purchase	POST	Purchase policy	API Key
/api/v2/policies	GET	List active policies	None

11.2 SKILL File

Lumina provides a SKILL file (LUMINA-SKILL.txt) loadable into any AI agent framework (Claude, ChatGPT, ElizaOS, LangChain, or custom agents). The SKILL contains complete protocol documentation, API endpoints, contract addresses, and usage examples.

11.3 Integration Channels

- **REST API Direct:** Pure M2M via HTTP.
- **SKILL File:** Natural language agent integration.
- **ElizaOS Plugin:** Native framework plugin (planned).
- **LangChain Tools:** Python tools package (planned).
- **Farcaster Frames:** Social-native purchasing (planned).

12. Legal & Regulatory Framework

12.1 Product Classification

Lumina Protocol products are structured as **parametric derivative contracts**, not traditional insurance policies. Key distinctions: (1) payouts are triggered by objective, verifiable on-chain data — not by demonstrated loss, (2) there is no claims adjustment process, (3) no insurable interest requirement exists, and (4) the protocol operates as a decentralized smart contract system without a central issuing entity.

This classification positions Lumina outside the scope of traditional insurance regulation in most jurisdictions, similar to how weather derivatives and catastrophe bonds operate in traditional finance.

12.2 Jurisdictional Considerations

The protocol is deployed on a public blockchain (Base L2) and accessible globally. Users interact directly with smart contracts through a permissionless API. The founding team has background in the Argentine insurance industry (PAS — Productores Asesores de Seguros) and maintains awareness of regulatory developments across jurisdictions.

12.3 Consumer Protections

- **Transparency:** All contract code is open-source and verified on BaseScan.
- **Deterministic pricing:** Premium formulas are published and verifiable before purchase.
- **Automatic settlement:** No discretionary denial of claims — triggers are objective.
- **Capital segregation:** Four isolated vaults prevent cross-contamination of risk pools.
- **Governance delay:** 48-hour timelock on all administrative changes.

13. Governance

Component	Configuration
Admin	Gnosis Safe (2-of-3 multisig on Base)
Timelock	48-hour mandatory delay for all admin actions
Upgrade Authority	TimelockController — no instant upgrades possible
Oracle Management	Signer rotation through timelock
Emergency Pause	Available through timelock (48h delay)

Future governance may include a decentralized governance token. The current model prioritizes security and operational agility appropriate for an early-stage protocol.

14. Roadmap

Phase	Milestone	Status
1	Core contracts + 4 vaults + 4 shields	Complete
2	API + Dashboard + Tutorial + SKILL	Complete
3	Security review (dual-auditor methodology)	Complete
4	Key rotation + Multisig Oracle + TimelockController	Complete
5	Production deploy (real USDC + Aave V3)	Complete
6	Seed liquidity + first production policies	In Progress
7	ElizaOS plugin + LangChain tools	Planned
8	Chainlink Automation for BSS/Depeg triggers	Planned
9	On-chain indexer (The Graph / Ponder)	Planned
10	Tier 1 external security audit	Planned
11	Governance token + secondary market	Future

15. Contract Addresses

All contracts deployed and verified on Base L2 (Chain 8453).

Contract	Address
CoverRouter	0xd5f8678A0F2149B6342F9014CCe6d743234Ca025
PolicyManager	0xCCA07e06762222AA27DEd58482DeD3d9a7d0162a
LuminaOracle	0xB52BB8B09Df13dB2D244746688C14A720ceE4C09
PhalaVerifier	0x468b9D2E9043c80467B610bC290b698ae23adb9B
VolatileShort Vault	0xbd44547581b92805aAECc40EB2809352b9b2880d
VolatileLong Vault	0xFee5d6DAdA0A41407e9EA83d4F357DA6214Ff904
StableShort Vault	0x429b6d7d6a6d8A62F616598349Ef3C251e2d54fC
StableLong Vault	0x1778240E1d69BEC8c0988BF1948336AA0Ea321c
BSS Shield	0x54CDc21DEDA49841513a6a4A903dc0A0a9e7844e
Depeg Shield	0x71DBcE71AA36370f7357F6D8E0c8ba96343C8306
IL Shield	0x4196f2Cc92C5c4141a34f9a28f23236446E3C4E0
Exploit Shield	0xaE29Fc3e5f0DedC968cE2dA2A2F3ccB98397b38C
TimelockController	0xd0De5D53dCA2D96cdE7FAf540BA3f3a44fdB747a
Gnosis Safe	0xa17e8b7f985022BC3c607e9c4858A1C264b33cFD
USDC	0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913

16. References

- [1] Lumina Protocol GitHub — <https://github.com/org-lumina/LUMINA-PROTOCOL>
- [2] Lumina Protocol Website — <https://www.lumina-org.com>
- [3] SKILL File — <https://www.lumina-org.com/LUMINA-SKILL.txt>
- [4] API Documentation — <https://github.com/org-lumina/LUMINA-PROTOCOL/docs/API-REFERENCE.md>
- [5] Chainlink Price Feeds — <https://docs.chain.link/data-feeds/price-feeds>
- [6] Aave V3 — <https://docs.aave.com/>
- [7] Base L2 — <https://docs.base.org/>
- [8] OpenZeppelin — <https://docs.openzeppelin.com/contracts/>
- [9] EIP-712 — <https://eips.ethereum.org/EIPS/eip-712>
- [10] Gnosis Safe — <https://safe.global/>
- [11] Foundry — <https://book.getfoundry.sh/>

© 2026 Lumina Protocol Labs. All rights reserved.
labs@lumina-org.com | <https://www.lumina-org.com>